

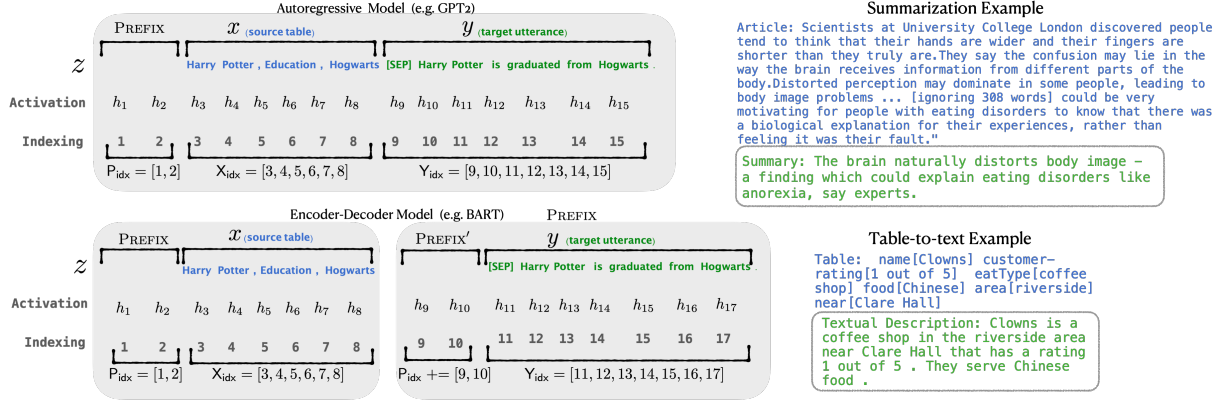


Figure 2: An annotated example of prefix-tuning using an autoregressive LM (top) and an encoder-decoder model (bottom). The prefix activations $\forall i \in P_{idx}, h_i$ are drawn from a trainable matrix P_θ . The remaining activations are computed by the Transformer.

4.1 Intuition

Based on intuition from prompting, we believe that having a proper context can steer the LM without changing its parameters. For example, if we want the LM to generate a word (e.g., Obama), we can prepend its common collocations as context (e.g., Barack), and the LM will assign much higher probability to the desired word. Extending this intuition beyond generating a single word or sentence, we want to find a context that steers the LM to solve an NLG task. Intuitively, the context can influence the encoding of x by guiding what to extract from x ; and can influence the generation of y by steering the next token distribution. However, it's non-obvious whether such a context exists. Natural language task instructions (e.g., "summarize the following table in one sentence") might guide an expert annotator to solve the task, but fail for most pretrained LMs.² Data-driven optimization over the discrete instructions might help, but discrete optimization is computationally challenging.

Instead of optimizing over discrete tokens, we can optimize the instruction as continuous word embeddings, whose effects will be propagated upward to all Transformer activation layers and rightward to subsequent tokens. This is strictly more expressive than a discrete prompt which requires matching the embedding of a real word. Meanwhile, this is less expressive than intervening all layers of the activations (§7.2), which avoids long-range dependencies and includes more tunable parameters. Prefix-tuning, therefore, optimizes all layers of the prefix.

²In our preliminary experiments, GPT-2 and BART fail in this setting; the only exception is GPT-3.

4.2 Method

Prefix-tuning prepends a prefix for an autoregressive LM to obtain $z = [\text{PREFIX}; x; y]$, or prepends prefixes for both encoder and decoder to obtain $z = [\text{PREFIX}; x; \text{PREFIX}'; y]$, as shown in Figure 2. Here, P_{idx} denotes the sequence of prefix indices, and we use $|P_{idx}|$ to denote the length of the prefix.

We follow the recurrence relation in equation (1), except that the prefix are *free* parameters. Prefix-tuning initializes a trainable matrix P_θ (parametrized by θ) of dimension $|P_{idx}| \times \dim(h_i)$ to store the prefix parameters.

$$h_i = \begin{cases} P_\theta[i, :], & \text{if } i \in P_{idx}, \\ \text{LM}_\phi(z_i, h_{<i}), & \text{otherwise.} \end{cases} \quad (3)$$

The training objective is the same as equation (2), but the set of trainable parameters changes: the language model parameters ϕ are fixed and the prefix parameters θ are the only trainable parameters.

Here, h_i (for all i) is a function of the trainable P_θ . When $i \in P_{idx}$, this is clear because h_i copies directly from P_θ . When $i \notin P_{idx}$, h_i still depends on P_θ , because the prefix activations are always in the left context and will therefore affect any activations to its right.

4.3 Parametrization of P_θ

Empirically, directly updating the P_θ parameters leads to unstable optimization and a slight drop in performance.³ So we reparametrize the matrix $P_\theta[i, :] = \text{MLP}_\theta(P'_\theta[i, :])$ by a smaller matrix (P'_θ) composed with a large feedforward neural network (MLP_θ). Note that P_θ and P'_θ has the same rows

³We find in preliminary experiments that directly optimizing the prefix is very sensitive to the learning rate and initialization.

dimension (i.e. the prefix length), but different columns dimension.⁴ Once training is complete, these reparametrization parameters can be dropped, and only the prefix (P_θ) needs to be saved.

5 Experimental Setup

5.1 Datasets and Metrics

We evaluate on three standard neural generation datasets for the table-to-text task: E2E (Novikova et al., 2017), WebNLG (Gardent et al., 2017), and DART (Radev et al., 2020). The datasets are ordered by increasing complexity and size. E2E only has 1 domain (i.e. restaurant reviews); WebNLG has 14 domains, and DART is open-domain, using open-domain tables from Wikipedia.

The E2E dataset contains approximately 50K examples with 8 distinct fields; it contains multiple test references for one source table, and the average output length is 22.9. We use the official evaluation script, which reports BLEU (Papineni et al., 2002), NIST (Belz and Reiter, 2006), METEOR (Lavie and Agarwal, 2007), ROUGE-L (Lin, 2004), and CIDEr (Vedantam et al., 2015).

The WebNLG (Gardent et al., 2017) dataset consists of 22K examples, and the input x is a sequence of (subject, property, object) triples. The average output length is 22.5. In the training and validation splits, the input describes entities from 9 distinct DBpedia categories (e.g., Monument). The test split consists of two parts: the first half contains DB categories seen in training data, and the second half contains 5 unseen categories. These unseen categories are used to evaluate extrapolation. We use the official evaluation script, which reports BLEU, METEOR and TER (Snover et al., 2006).

DART (Radev et al., 2020) is an open domain table-to-text dataset, with similar input format (entity-relation-entity triples) as WebNLG. The average output length is 21.6. It consists of 82K examples from WikiSQL, WikiTableQuestions, E2E, and WebNLG and applies some manual or automated conversion. We use the official evaluation script and report BLEU, METEOR, TER, MoverScore (Zhao et al., 2019), BERTScore (Zhang et al., 2020b) and BLEURT (Sellam et al., 2020).

For the summarization task, we use the XSUM (Narayan et al., 2018) dataset, which is an abstrac-

tive summarization dataset on news articles. There are 225K examples. The average length of the articles is 431 words and the average length of the summaries is 23.3. We report ROUGE-1, ROUGE-2 and ROUGE-L.

5.2 Methods

For table-to-text generation, we compare prefix-tuning with three other methods: fine-tuning (FINE-TUNE), fine-tuning only the top 2 layers (FT-TOP2), and adapter-tuning (ADAPTER).⁵ We also report the current state-of-the-art results on these datasets: On E2E, Shen et al. (2019) uses a pragmatically informed model without pretraining. On WebNLG, Kale (2020) fine-tunes T5-large. On DART, no official models trained on this dataset version are released.⁶ For summarization, we compare against fine-tuning BART (Lewis et al., 2020).

5.3 Architectures and Hyperparameters

For table-to-text, we use GPT-2_{MEDIUM} and GPT-2_{LARGE}; the source tables are linearized.⁷ For summarization, we use BART_{LARGE},⁸ and the source articles are truncated to 512 BPE tokens.

Our implementation is based on the Hugging Face Transformer models (Wolf et al., 2020). At training time, we use the AdamW optimizer (Loshchilov and Hutter, 2019) and a linear learning rate scheduler, as suggested by the Hugging Face default setup. The hyperparameters we tune include the number of epochs, batch size, learning rate, and prefix length. Hyperparameter details are in the appendix. A default setting trains for 10 epochs, using a batch size of 5, a learning rate of $5 \cdot 10^{-5}$ and a prefix length of 10. The table-to-text models are trained on TITAN Xp or GeForce GTX TITAN X machines. Prefix-tuning takes 0.2 hours per epochs to train on 22K examples, whereas fine-tuning takes around 0.3 hours. The summarization models are trained on Tesla V100 machines, taking 1.25h per epoch on the XSUM dataset.

At decoding time, for the three table-to-text datasets, we use beam search with a beam size of 5. For summarization, we use a beam size of 6

⁵Same implementation as Lin et al. (2020).

⁶The official benchmark model is trained on v1.0.0 while the release dataset is v1.1.1.

⁷In comparison with natural language utterances, the linearized table is in an unnatural format, which might be challenging for pretrained LMs.

⁸We didn't include GPT-2 results for summarization because in our preliminary experiment, fine-tuning GPT-2 significantly underperforms fine-tuning BART on XSUM.

⁴ P_θ has a dimension of $|P_{idx}| \times \dim(h_i)$ while P_θ has a dimension of $|P_{idx}| \times k$, where we choose $k = 512$ for table-to-text and 800 for summarization. MLP_θ maps from dimension k to $\dim(h_i)$

and length normalization of 0.8. Decoding takes 1.2 seconds per sentence (without batching) for table-to-text, and 2.6 seconds per batch (using a batch size of 10) for summarization.

6 Main Results

6.1 Table-to-text Generation

We find that adding only 0.1% task-specific parameters,⁹ prefix-tuning is effective in table-to-text generation, outperforming other lightweight baselines (ADAPTER and FT-TOP2) and achieving a comparable performance with fine-tuning. This trend is true across all three datasets: E2E, WebNLG,¹⁰ and DART.

For a fair comparison, we match the number of parameters for prefix-tuning and adapter-tuning to be 0.1%. Table 1 shows that prefix-tuning is significantly better than ADAPTER (0.1%), attaining 4.1 BLEU improvement per dataset on average. Even when we compare with fine-tuning (100%) and adapter-tuning (3.0%), which update significantly more parameters than prefix-tuning, prefix-tuning still achieves results comparable or better than those two systems. This demonstrates that prefix-tuning is more Pareto efficient than adapter-tuning, significantly reducing parameters while improving generation quality.

Additionally, attaining good performance on DART suggests that prefix-tuning can generalize to tables with diverse domains and a large pool of relations. We will delve deeper into extrapolation performance (i.e. generalization to unseen categories or topics) in §6.4.

Overall, prefix-tuning is an effective and space-efficient method to adapt GPT-2 to table-to-text generation. The learned prefix is expressive enough to steer GPT-2 in order to correctly extract contents from an unnatural format and generate a textual description. Prefix-tuning also scales well from GPT-2_{MEDIUM} to GPT-2_{LARGE}, suggesting it has the potential to scale to even larger models with a similar architecture, like GPT-3.

6.2 Summarization

As shown in Table 2, with 2% parameters, prefix-tuning obtains slightly lower performance than fine-

tuning (36.05 vs. 37.25 in ROUGE-L). With only 0.1% parameters, prefix-tuning underperforms full fine-tuning (35.05 vs. 37.25). There are several differences between XSUM and the three table-to-text datasets which could account for why prefix-tuning has comparative advantage in table-to-text: (1) XSUM contains 4x more examples than the three table-to-text datasets on average; (2) the input articles are 17x longer than the linearized table input of table-to-text datasets on average; (3) summarization might be more complex than table-to-text because it requires reading comprehension and identifying key contents from an article.

6.3 Low-data Setting

Based on the results from table-to-text (§6.1) and summarization (§6.2), we observe that prefix-tuning has a comparative advantage when the number of training examples is smaller. To construct low-data settings, we subsample the full dataset (E2E for table-to-text and XSUM for summarization) to obtain small datasets of size {50, 100, 200, 500}. For each size, we sample 5 different datasets and average over 2 training random seeds. Thus, we average over 10 models to get an estimate for each low-data setting.¹¹

Figure 3 (right) shows that prefix-tuning outperforms fine-tuning in low-data regimes by 2.9 BLEU on average, in addition to requiring many fewer parameters, but the gap narrows as the dataset size increases.

Qualitatively, Figure 3 (left) shows 8 examples generated by both prefix-tuning and fine-tuning models trained on different data levels. While both methods tend to undergenerate (missing table contents) in low data regimes, prefix-tuning tends to be more faithful than fine-tuning. For example, fine-tuning (100, 200)¹² falsely claims a low customer rating while the true rating is average, whereas prefix-tuning (100, 200) generates a description that is faithful to the table.

6.4 Extrapolation

We now investigate extrapolation performance to unseen topics for both table-to-text and summarization. In order to construct an extrapolation setting, we split the existing datasets so that training and test cover different topics. For table-to-text, the

⁹250K for E2E, 250K for WebNLG, and 500K for DART vs. 345M GPT-2 parameters.

¹⁰The S,U,A columns in WebNLG represents SEEN, UNSEEN, and ALL respectively; SEEN categories appear at training time; UNSEEN categories only appears at test time; and ALL is the combination of the two.

¹¹We also sample a dev split (with dev size = 30% × training size) for each training set. We use the dev split to choose hyperparameters and do early stopping.

¹²The number in the parenthesis refers to the training size.